



Programmieren

Falk Morgenstern
Leon Johann Brettin

Foliensatz basierend auf Unterlagen von Prof. Huhn, Prof. Meyer und weiteren.
Nur zur Verwendung in der Vorlesung „Programmieren“ an der Ostfalia.
Keine Weitergabe.



Programmieren

8. Collections



Programmieren

8. Collections

8.1 Collection Typen

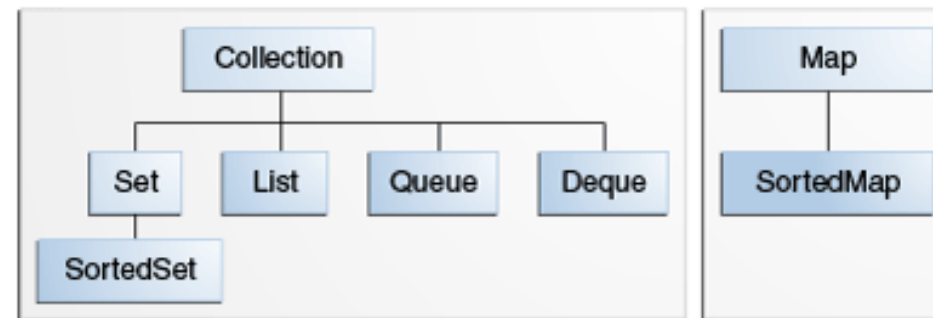


Was sind Collection Typen?

- Collection meint „Sammlung“ von Objekten
- Verschiedene grundlegende Datenstrukturen zur Bildung von Sammlungen bekannt
- Viele Datenstrukturen werden immer wieder verwendet
 - **Listen** in Form von Arrays oder verketteten Listen
 - **Mengen** von Elementen organisiert für schnellen Zugriff
 - **Abbildungen**: Paare aus Schlüsselementen und zugeordneten Werten (Beispiel: Telefonbuch)
- Java bietet Implementierungen hierfür in der Klassenbibliothek
- Damit sie für beliebige Typen von Elementen verwendbar sind, sind sie als generische Klassen implementiert.

Interfaces Collection Typen (Collection, Map)

- Die Struktur der Collection-Typen ist durch die folgenden Interfaces vorgegeben



<https://docs.oracle.com/javase/tutorial/collections/interfaces/index.html>

- Zusätzlich gibt es Templates in Form abstrakter Klassen, die für eigene Implementierung genutzt werden können.
- Die ganze Wahrheit findet sich in der Java API

<https://docs.oracle.com/en/java/javase/17/docs/api/java.base/java/util/Collection.html>



Methoden des Interface Collection<E>

- Eine Collection steht für eine Sammlung von Elementen. Das Interface Collection enthält u.a. folgenden Methoden:
- `boolean add(E obj)`
- `boolean remove(Object obj)`
- `void clear()`
- `boolean contains(Object obj)`
- `int size()`
- `Iterator<E> iterator()`
- ...
- Collection-Typen sind generische Typen. Wir können Sammlungen von beliebigen Referenztypen anlegen.



Beispiele für Collection-Typen

- Mögliche Spezialisierungen der Interfaces `Collection` und `Map`, die wir hier betrachten, sind:
 - Listen (`List`): Speicherung von Elementen in der Reihenfolge, in der sie hinzugefügt wurden (Basis für `LinkedList`, `ArrayList`, ...). Elemente können über einen Index angesprochen werden.
 - Mengen (`Set`): Sammlungen ohne Duplikate; die Reihenfolge der Ablage wird nicht gespeichert.
 - Geordnete Mengen (`SortedSet`): Sammlungen ohne Duplikate, auf deren Elementen eine Ordnung existiert (implementieren `Comparable<E>`)
 - Abbildungen (`Map`): Effiziente Speicherung von Schlüssel-Wert-Paaren, z.B. Artikelnummer, Bezeichnung o.ä.



Programmieren

8. Collections

8.2 Listen

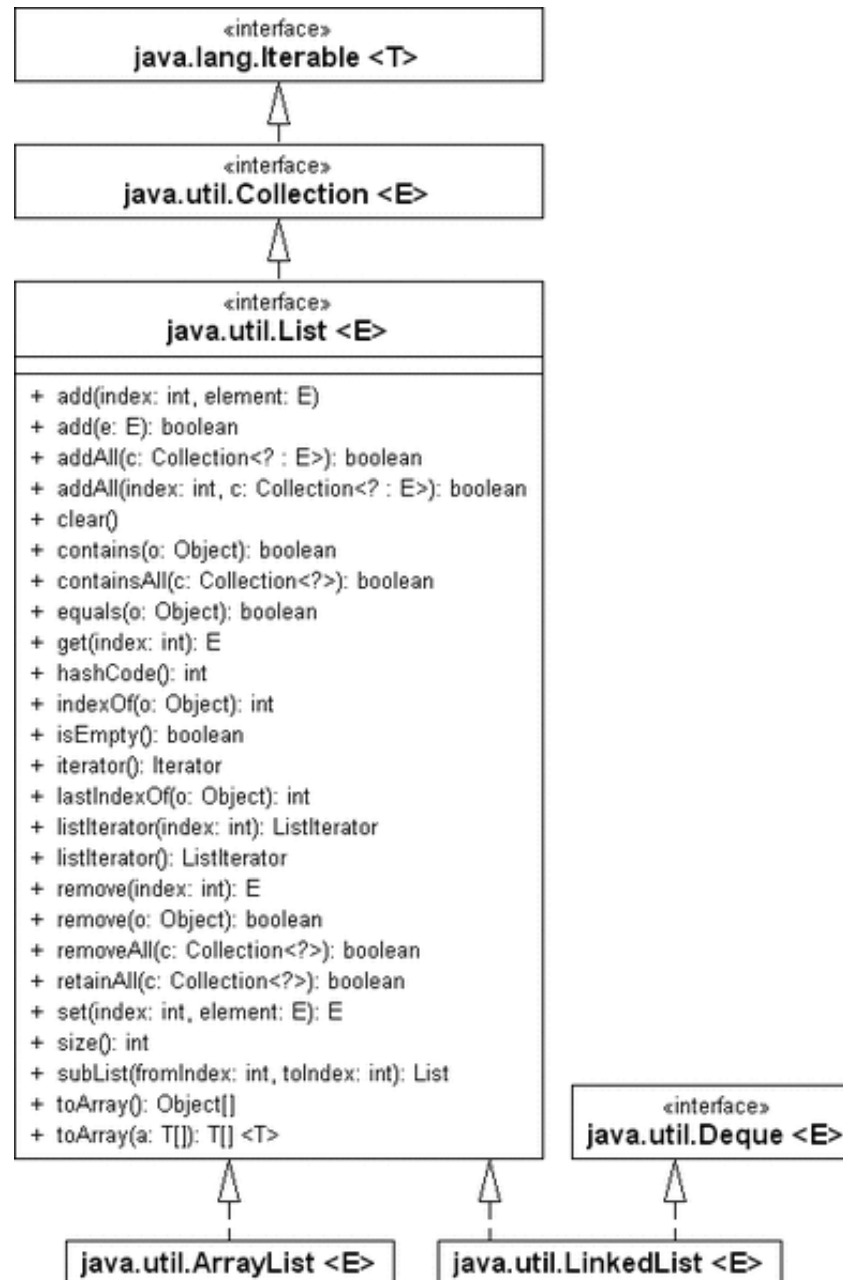


Listen

- im Interface List gibt es (zusätzlich) Methoden für:
 - `void add(int index, T element)`
 - `T get(int index)`
 - `T remove(int index)`
 - `int indexOf(Object o)`
 - `ListIterator<T> listIterator()`
 - ...



Listen





Klassen ArrayList, LinkedList

- **ArrayList**

- Realisierung einer Liste als Array.
- Gut, wenn Zugriff über Index verwendet werden soll.
- Erweitern des Arrays durch Umkopieren in größeres Array.

- **LinkedList**

- Realisierung einer Liste als doppelt verkettete Liste, d.h. jedes Listenelement besitzt Verweis auf Vorgänger und Nachfolger.
- Gut, wenn oft Elemente eingefügt werden sollen. Besitzt deshalb zusätzliche Methoden `addFirst()`, `addLast()`, `removeFirst()`, `removeLast()`.
- Zugriff über Index ist langsamer.



Beispiel: List, ArrayList

```
import java.util.ArrayList;
import java.util.Arrays;
import java.util.List;

public class Listen {

    public static void main(String[] args) {
        ArrayList<String> list = new ArrayList<String>(); // erzeugen
        list.add("Anton"); // Elemente mit add hinzufügen
        list.add("Berta");
        list.add(1, "Caesar");
        System.out.println("\nGroesse der Liste: " + list.size());
        for (int i = 0; i < list.size(); i++) { // Liste ausgeben
            System.out.println(list.get(i));
        }
        if (list.contains("Caesar")) { // "Caesar" loeschen
            list.remove("Caesar");
        }

        Integer[] ar = { 1, 2, 3, 4 };
        List<Integer> l1 = Arrays.asList(ar); // List aus Array erzeugen
        List<Integer> l2 = new ArrayList<>(Arrays.asList(ar));
        ar[0] = 42; // Achtung, Änderungen auch sichtbar in l1
        System.out.println(l1.get(0)); // 42
        System.out.println(l2.get(0)); // 1
    }
}
```





Ausgabe mit Iterator-Objekt

- Ein Iterator-Objekt bietet die folgenden Methoden
- `boolean hasNext()`
 - `true`, wenn es mindestens ein weiteres Element in der Collection gibt.
- `T next()`
 - gibt das nächste Element der Collection zurück
- `void remove()`
 - entfernt das Element, das beim letzten Aufruf von `next()` zurückgegeben wurde (nicht von allen Iteratoren unterstützt)

```
java.util.Iterator<String> it = list.iterator();  
while (it.hasNext()) {  
    String s = it.next();  
    System.out.println(s);  
}
```





Ausgabe mit Iterator-Objekt

- Neben dem allgemeinen Iterator gibt es auch einen speziellen Iterator für Listen `ListIterator`, der z.B. über `nextIndex()` auch den Zugriff auf die Indizes der Elemente ermöglicht.
- `int nextIndex()`
 - Index des Elements, das beim nächsten Aufruf von `next()` zurückgegeben wird, analog für `previous`
- `boolean hasPrevious()`
 - `true`, wenn es mindestens ein vorangehendes Element in der Liste gibt.
- `T previous()`
 - gibt das vorangehende Element der Liste zurück.

```
ListIterator<String> lit = list.listIterator(list.size());  
while (lit.hasPrevious()) {  
    System.out.println(lit.previousIndex());  
    System.out.println(lit.previous());  
}
```





Iterieren mit for-Schleife

- Für Arrays haben Sie bereits die Iterator-Form der for-Schleife kennengelernt.
- Mithilfe dieser for-Schleife können wir direkt über Collections iterieren
- Funktioniert für alle Typen, die das Interface `Iterable` implementieren, also einen Iterator erzeugen können
- Collection-Klassen implementieren das Interface `Iterable`

```
for (String s : list) {  
    System.out.println(s);  
}
```





Programmieren

8. Collections

8.3 Mengen



Mengen

- Mengen (Set)
 - Sammlungen ohne Duplikate, Reihenfolge der Ablage wird nicht gespeichert.
- Ziel
 - Möglichst effiziente Suche, ob ein Element in der Menge ist.
- Frage
 - Welche Möglichkeiten kennen Sie, eine Menge von Objekten für eine möglichst effiziente Suche optimiert abzuspeichern?



Mengen

- **Klasse HashSet**
 - Reihenfolge der Speicherung ist uninteressant,
 - keine Duplikate und
 - es soll keine spezielle Ordnung auf den Elementen abgebildet werden
- **Klasse TreeSet**
 - Reihenfolge der Speicherung ist uninteressant,
 - keine Duplikate und
 - die Elemente der Menge besitzen eine Ordnung. Es soll möglich sein, sie gemäß der Ordnung auszulesen.



Mengen

- HashSet
 - Speicherung in Hash-Tabelle, dabei Verkettung der Überläufer bei Kollisionen.
 - Hash-Funktion ist durch Methode `hashCode()` der Elemente gegeben. D.h. `hashCode()` muss geeignet implementiert sein.
- TreeSet
 - Speicherung durch einen balancierten binären Baum (realisiert als Rot-Schwarz-Baum).
 - Für die Elemente der Menge muss das Interface `Comparator` bereitgestellt werden: `TreeSet(Comparator<? super E> c)`



HashSet

- So wird ein Element `elem` vom Typ `T` in einem Objekt der Klasse `HashSet<T>` gespeichert (ausführlicher siehe Algorithmen und Datenstrukturen):
 - Die Hashtabelle ist als Array realisiert.
 - `elem.hashCode()` gibt den Index an, unter dem das Element in dem Array abgespeichert wird.
 - Da verschiedene Objekte vom Typ `T` den gleichen Hashwert haben können, sind die Einträge der Hashtabelle verkettete Listen.
 - D.h. `elem` wird an die verkettete Liste mit dem Array-Index `elem.hashCode()` angehängt.
- Solange die Hashfunktion gut und die Hashtabelle noch nicht zu voll ist, kann man in annähernd konstanter Zeit ein Element suchen.



HashSet Beispiel

```
java.util.HashSet<String> ort = new HashSet<>();
ort.add("Berlin");
ort.add("Hamburg");
ort.add("München");
ort.add("Berlin");

for (String s : ort) {
    System.out.println(s + " " + s.hashCode());
}

// z.B.
// Berlin 1986302914
// Hamburg -1938484676
// München -1652442035
```





Mengen

- Reihenfolge der Elemente irrelevant
- Der Zugriff auf einzelne Elemente mittels `get` ist nicht direkt möglich
 - Umwandlung in Array und per Index zugreifen
 - Umwandlung in Liste und per `get` zugreifen
- Normalerweise: Iterator nutzen
- Effizient ist die Suche nach Elementen

```
System.out.println(ort.contains("Berlin"));  
System.out.println(ort.contains("Hannover"));
```





Programmieren

8. Collections

8.4 Abbildungen



Abbildungen

- **Abbildungen** (Map)
 - Effiziente Speicherung von **Schlüssel-Wert-Paaren**
 - Jedem Schlüssel ist ein Element zugeordnet, deshalb bezeichnet man die Zuordnung als Abbildung.
- Frage
 - Überlegen Sie sich Beispiele für Schlüssel-Wert-Paare, was ist jeweils der Schlüssel, was der Wert?
- Realisierung durch Klassen `HashMap` (`Hashtable`) und `TreeMap`
- `HashMap`
 - Verwendet Hashcode des Schlüssels, um gesuchte Werte wiederzufinden
- `TreeMap`
 - Nach Schlüsseln aufsteigend sortiert (balancierter binärer Baum)



Abbildungen

- Das Interface `Map<K, V>` stellt unter anderem die folgenden Methoden bereit. Es werden zwei Typparameter `K` und `V` genutzt. `K` steht für Key und `V` für Value.
 - `V put(K key, V value)`: legt das Paar (`key`, `value`) in der Map ab
 - `V get(Object key)`: gibt den Wert zurück, der `key` zugeordnet ist
 - `V remove(Object key)`
 - `void clear()`
 - `int size()`
 - `boolean containsKey(Object key)`: Ist `key` vorhanden?
 - `boolean containsValue(Object value)`: Ist `value` vorhanden?
 - `Set<Key> keySet()`: Menge aller vorkommenden Schlüssel
 - `Collection<V> values()`: Collection aller vorkommenden Werte
 - `Set<Map.Entry<K, V>> entrySet()`: liefert ein Set von `Map.Entry`-Objekten, die Key (Schlüssel) und Value (Wert) eines Eintrags enthalten



Abbildungen: Beispiel

```
java.util.HashMap<String, Integer> phoneBook = new HashMap<>();  
phoneBook.put("Müller", 12345);  
phoneBook.put("Schulz", 234346);  
phoneBook.put("Kunz", 23123);  
phoneBook.put("Müller", 3333);  
  
for (String k : phoneBook.keySet()) {  
    System.out.println(k + " " + phoneBook.get(k));  
}  
  
// Kunz 23123  
// Müller 3333  
// Schulz 234346  
  
if (phoneBook.containsKey("Kunz")) {  
    phoneBook.remove("Kunz");  
}  
  
System.out.println(phoneBook.toString()); // {Müller=3333, Schulz=234346}
```





Abbildungen: Beispiel

```
Set<Entry<String, Integer>> entrySet = phoneBook.entrySet();
Iterator<Entry<String, Integer>> it = entrySet.iterator();
while (it.hasNext()) {
    Entry<String, Integer> e = it.next();
    System.out.println(e);
}
// Müller=3333
// Schulz=234346
```





Übung

- Entwerfen Sie eine Kundendatenbank in Java mit Hilfe der Java Collections.
- Welche Collection ist geeignet, wenn keine Duplikate (gleicher Kundenname) in der Datenbank gespeichert werden soll und Sie Kunden anhand ihres Namens auffinden möchten?
- Welche Methoden müssen Sie überschreiben, damit die Duplikatprüfung (gleicher Name) funktioniert?
- Implementieren Sie die Kundendatenbank in Java. Hier benötigen Sie auch die Klasse Kunde mit geeigneten Attributen.